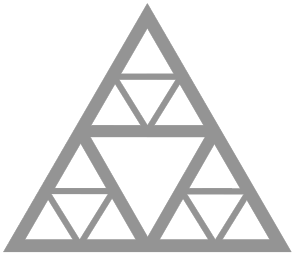


ÉCOLE NATIONALE DES PONTS ET CHAUSSÉES

Drug Design

Semester project report IMI

Gaspard Beaudouin
Maxime Muhlethaler
Titouan Pottier

Under the supervision of Paraskevi Gkeka, Gabriel Stolz, Tony Levièvre and Régis Santet
Sanofi
&
CERMICS, Ecole des Ponts

Table des matières

1 Context and Motivation

1.1 AI in drug discovery

Artificial Intelligence has made significant breakthroughs in the field of drug discovery, revolutionizing the way new medicines are developed. Drug discovery is a complex and multifaceted process that requires the integration of various scientific disciplines and techniques. AI is now an essential tool in this field, streamlining the discovery process and accelerating the development of new drugs. The different step are :

— **De Novo Design** : AI techniques, are used in de novo drug design to identify and generate candidate drugs from scratch. By leveraging molecular knowledge and advanced algorithms such as generative models, including generative adversarial networks (GANs) and variational autoencoders (VAEs), AI can design novel compounds that could potentially serve as effective drugs.

— **Target Structure Prediction** : Understanding the structure of biological targets is crucial for drug discovery. AI models based on deep learning like AlphaFold and RoseTTAFold can predict the three-dimensional structure of proteins and other molecular targets based on their amino acid sequences.

— **Drug-Target Interaction** : AI models can assess the interactions between drugs and their targets. By analyzing the molecular properties of both drugs and targets, AI can predict how well a drug will bind to its target.

— **Drug-Target Binding Affinity** : AI can also predict the binding affinity between drugs and their targets. This involves estimating how strongly a drug molecule will bind to its target, which is a critical factor in determining the efficacy of a drug. AI models use various features of the drug and target to predict binding affinity.

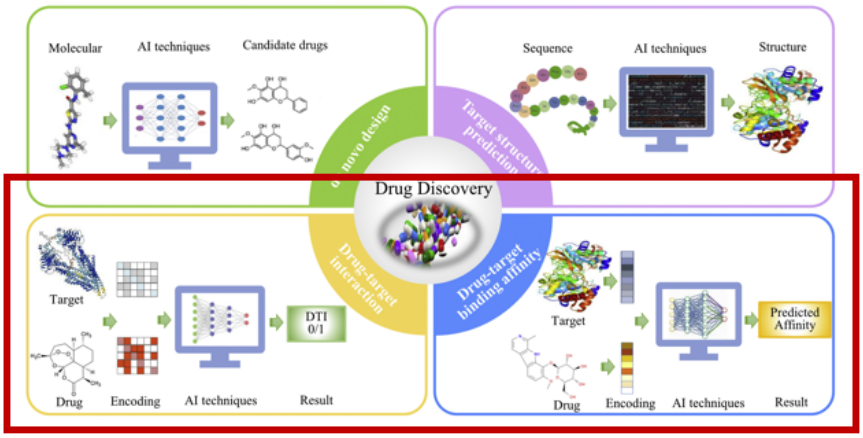


FIGURE 1 – Drug Discovery : a complex and multifaceted process

In this project, we have focused on the 2 last step of drug discovery, which can be summerized into molecular docking known as a computational technique vital for predicting the binding interactions between small molecules (ligands) and target proteins and the ligand's pose including its position and orientation. Molecular docking is essential for determining the potential affinity and stability of the interaction between a ligand and its protein, facilitating the identification of compounds that can effectively bind to **protein sites linked to diseases**. It is crucial for the drug discovery process.

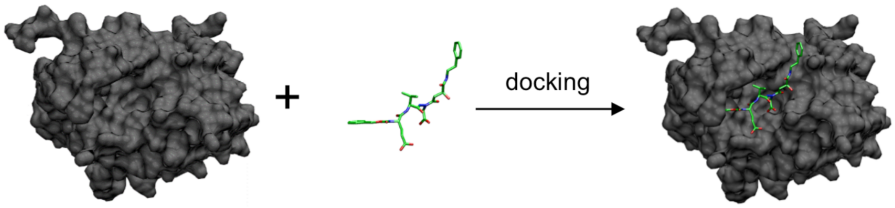


FIGURE 2 – Molecular Docking

1.2 Motivation of molecular docking and studied case

We want to motivate why molecular docking is crucial in drug discovery. To underlines that, we have explored in this project the example of the BCR-ABL protein and a ligand.

BCR-ABL is a chimeric protein resulting from the fusion of two genes, the BCR gene and the ABL gene, due to a genetic translocation (exchange of segments between two chromosomes). This genetic fusion leads to the expression of an abnormal protein with constitutive tyrosine kinase activity, meaning excessive enzymatic activity that promotes uncontrolled cell growth.

This protein is involved in the pathogenesis of chronic myeloid leukemia and other types of blood cancer. Treating cancers associated with BCR-ABL often involves targeting the enzymatic activity of the protein.

The ligand is the compound HG-7-85-01, a small molecule that interacts with the BCR-ABL protein. This ligand is a tyrosine kinase inhibitor capable of binding to the protein and blocking its enzymatic activity.

The study of this kind of protein-ligand interaction is motivated by several fact :
- **Understanding Interactions :** By using DiffDock to simulate and visualize the interaction between the BCR-ABL protein and the ligand, you can better understand how the ligand binds to the protein and how it inhibits its activity.
- **Ligand Optimization :** By examining how the ligand binds to the protein, you can identify possibilities for optimizing the ligand's chemical structure to improve its efficacy, selectivity, and reduce side effects.
- **Development of More Effective Therapies :** A detailed understanding of the interaction between BCR-ABL and the ligand allows for the development of more targeted therapies to treat cancers associated with BCR-ABL, potentially leading to more effective and better-tolerated treatments for patients.
- **Experimental Validation :** Docking simulations performed with DiffDock can guide laboratory experiments, suggesting the most promising compounds and targets for in vitro and in vivo testing.

1.3 Generative Models in drug discovery

Moreover, we have focused in this project on generative models that are know to achieve the best results. These are the main reasons :
- Generative models can simulate the entire docking process, considering multiple possible poses and interactions between the ligand and protein, while regression models, on the other hand, typically focus on predicting specific outcomes, such as binding affinity, based on a fixed set of molecular features.
- Generative models can explore a broader range of conformations and poses, leading to a more thorough search for optimal binding configurations.
- Generative models can generate multiple potential docking poses and assess their quality

and likelihood, providing a richer set of outputs for researchers to evaluate, while regression models usually provide a single predicted value (e.g., binding affinity) without offering insight into possible alternative poses.

— Generative models can incorporate dynamic and stochastic elements into their predictions, allowing them to model the inherent uncertainties and variations in molecular interactions, while regression models tend to provide deterministic outputs, which may oversimplify complex docking scenarios.

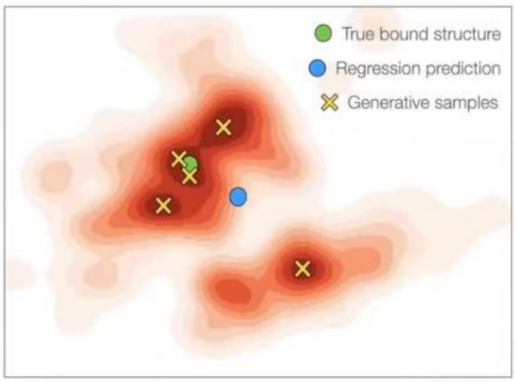


FIGURE 3 – Difference on Molecular Docking between Regression and Generative models

During this project, we explore two main questions : how can generative models be used in molecular docking, and how effective are diffusion models in enhancing the accuracy and efficiency of molecular docking ?

We will begin with an overview of diffusion models and score-based models. Next, we will examine how these models are concretely applied in the molecular docking software DiffDock. Finally, we will analyze some results from DiffDock and compare them with other molecular docking models.

2 Diffusion Models

Diffusion models work by gradually transforming an initial data distribution (noise) into a target data distribution, through an iterative process. This process is guided by models that **learn to reverse these diffusions**, allowing the **generation of samples from noise** by following the **inverse steps of diffusion.**

In this part, we will use mainly the formalism of L. Weng [4].

2.1 Diffusion models

2.1.1 Theory

Forward Diffusion process

Given a data sampled from a real data distribution $x_0 \sim q(x)$, let us define a forward diffusion process in which we add small amount of Gaussian noise to the sample in steps, producing a sequence of noisy samples . Here our data will be images to make it clear. The step sizes are controlled by a variance schedule β_t.

Given an image x_{t-1}, we sample x_t according to this distribution :

$$q\left(x_t \mid x_{t-1}\right) := \mathcal{N}\left(x_t; \sqrt{1-\beta_t} x_{t-1}, \beta_t I\right)$$

hence we have :

$$x_t = \sqrt{1-\beta_t} x_{t-1} + \sqrt{\beta_t} z_{t-1} \qquad \text{where} \quad z_{t-1} \sim \mathcal{N}(0, I)$$

However, we can show that we can sample directly a noisy image x_t only from x_0, so in only one step because :

$$q\left(x_t \mid x_0\right) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, \left(1-\overline{\alpha_t}\right) I\right) \qquad \text{where } \bar{\alpha}_t = \prod_{i=1}^{t} \left(1-\beta_i\right)$$

hence :

$$\boxed{x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} z_t} \tag{1}$$

Reverse Diffusion process

It is noteworthy that the reverse conditional probability is tractable when conditioned on x_0 :

$$q\left(x_{t-1} \mid x_t, x_0\right) = \mathcal{N}\left(x_{t-1}; \tilde{\mu}_t\left(x_t, x_0\right), \tilde{\beta}_t I\right)$$

$$\tilde{\mu}_t\left(\mathbf{x_t}, \mathbf{x}_0\right) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1-\bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}\left(1-\bar{\alpha}_{t-1}\right)}{1-\bar{\alpha}_t} \mathbf{x_t} \quad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$$

But what we want is to have $q\left(x_{t-1} \mid x_t\right)$ to denoise the image, and therefore no longer have the dependency on x_0.

But as $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} z_t$, we no longer have a dependency on x_0, but only on z_t : $\tilde{\mu}_t(\mathbf{x_t}, \mathbf{x_0}) = \tilde{\mu}_t(\mathbf{x_t}, \mathbf{z_t}) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x_t} - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \mathbf{z_t} \right)$

Eventually :

$$\boxed{q(x_{t-1} \mid x_t, z_t) = \mathcal{N}\left(x_{t-1}; \tilde{\mu}_t(x_t, z_t), \tilde{\beta}_t I\right)}$$

$$\tilde{\mu}_t(\mathbf{x_t}, \mathbf{z_t}) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x_t} - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \mathbf{z_t} \right) \approx \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \mathbf{z}_\theta(\mathbf{x_t}, \mathbf{t}) \right)$$

Thus, we will not attempt to denoise the image x_t directly towards the image x_{t-1}, but we will try to learn the noise z_t that was used to transition from x_0 to x_t.

The inverse process corresponds to

$$\boxed{\mathbf{x_t} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x_t} - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \mathbf{z_t} \right) + \tilde{\beta}_t \mathbf{z} \quad \text{où } \mathbf{z} \sim \mathcal{N}(0, I)} \tag{2}$$

We can use the variational lower bound to optimize the negative log-likelihood of $p_\theta(x_{t-1} \mid x_t)$. But empirically, the training is easier with a simplified objective :

$$L_t^{\text{simple}} = \mathbb{E}_{t \sim [1,T], \mathbf{x}_0, t} \left[\| \boldsymbol{z}_t - \boldsymbol{z}_\theta(\mathbf{x}_t, t) \|^2 \right]$$

During training, we will minimize this loss in order to predict, for a given image x_t and time t, the noise z_t that generated it from x_0.

Algorithm 1 : Training

Algorithm 1 Training algorithm

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ▷ Sample initial data
3: $t \sim \text{Uniform}(\{1, \dots, T\})$ ▷ Randomly choose a time step
4: $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ▷ Sample noise
5: Take gradient descent step on :

$$\nabla_z \left\| \boldsymbol{z} - \boldsymbol{z}_\theta\left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \boldsymbol{z}, t\right) \right\|^2$$

6: **until** converged

Algorithm 2 : Sampling

Algorithm 2 Sampling algorithm

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ▷ Initialize at the final step
for $t = T, \ldots, 1$ **do**
  if $t > 1$ **then**
    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ▷ Sample noise
  else
    $\mathbf{z} = \mathbf{0}$ ▷ No noise at the first step
  end if
  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{z}_\theta(\mathbf{x}_t, t)) + \sigma_t \mathbf{z}$
end for
return $\mathbf{x}_0$

2.1.2 Practical session : MNIST fashion

This practical session comes from CNRS Fidle course : "Diffusion model, text to image" Our dataset is composed of 60 000 32 x 32 images of fashion objects.

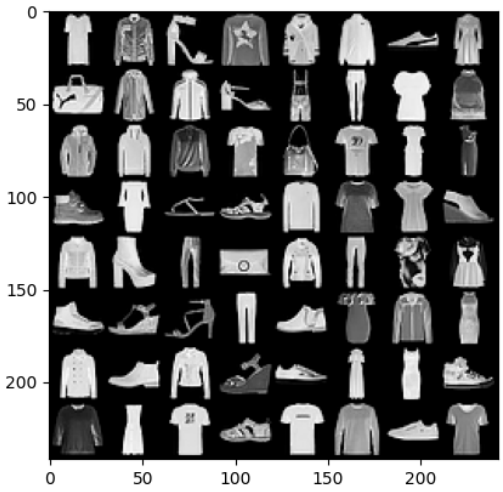


FIGURE 4 – fashion MNIST dataset

First, we want to make **forward diffusion process**. We create a function that will compute every betas of every steps (following a specific shedule). We will only create a function for the linear schedule (original DDPM) and the cosine schedule (improved DDPM).

After that, we define the function $q(x_t|x_0)$:

```
q_sample(constants_dict, batch_x0, batch_t, noise=None)
```

We can now visualize this process :

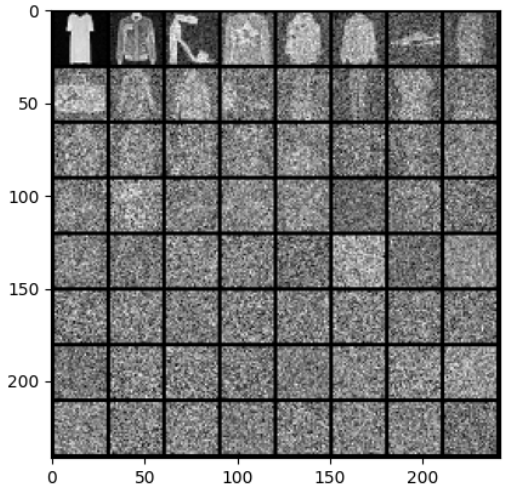


FIGURE 5 – Forward process

We then when to code the **reverse diffusion process**, which is made by a deep learning model : Unet with attention.

We also define a function to retrieve x_{t-1} from x_t and the predicted z_t :

`p_sample(constants_dict, batch_xt, predicted_noise, batch_t)`

For the **training**, we used 3 epochs and T=10000, an L1 smooth loss, and an ADAM optimizer, and then, we implemented Algorithm 1.

Eventually, We create the **sampling** function (Algorithm 2). Given trained model, it should generate all the images we want.

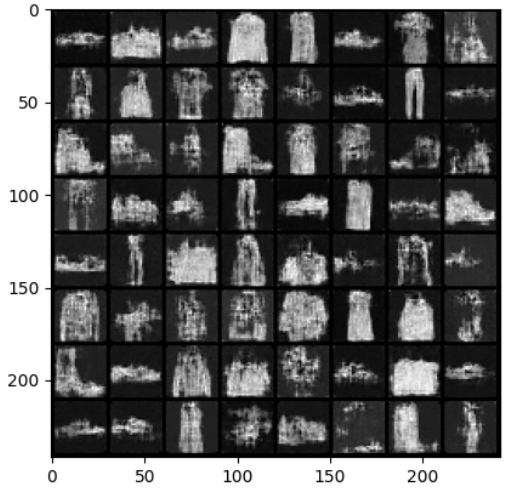


FIGURE 6 – Sampling results

2.2 Connection with score based diffusion models

We now want to make a link between diffusion models and score based diffusion models effectively used in our molecular docking software.

2.2.1 Theory

Score models learn score functions (gradients of logarithmic probability densities) over many data distributions perturbed by noise through **Stochastic Differential Equation (SDE)**, and then generate samples via the reverse SDE.[3]

Langevin dynamics is a concept from physics to model molecular systems. By modeling the score function instead of the density function, we can sidestep the difficulty of intractable normalizing constants. We use a stochastic gradient descent with the stochastic gradient Langevin dynamics to produce samples from a probability density $q(x)$ using only the gradients in a Markov chain of updates :

$$x^n = x^{n-1} + \frac{\delta}{2}\nabla_x log(q(x^{n-1})) + \sqrt{\delta}z^n \quad \text{where} \quad z^n \sim \mathcal{N}(0, I)$$

where δ is the step size. When $N \to \infty$, x^N equals to the true probability density $q(x)$.

Score-based generative modeling method can produce samples via Langevin dynamics using gradients of the data distribution estimated with score matching. The **score** of each sample $\mathbf{x}$'s density probability is defined as its gradient $\nabla_{\mathbf{x}} \log q(\mathbf{x})$. A score network $\mathbf{s}_\theta : \mathbb{R}^D \to \mathbb{R}^D$ is trained to estimate it, $\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log q(\mathbf{x})$.

However, as per the manifold hypothesis, the majority of data tends to cluster within a lower-dimensional manifold, despite appearing arbitrarily high-dimensional in observation. This phenomenon adversely affects score estimation, as data points may fail to cover the entirety of the space in regions of low data density. We can adress this issue by introducing small Gaussian noise to broaden the distribution of perturbed data, encompassing the entire space $\mathbb{R}^D$. This adjustment resulted in increased stability during the training of the score estimator network. Furthermore, we can improve the method by introducing noise of varying levels, training a noise-conditioned score network to concurrently estimate scores for all perturbed data across different noise levels.

Adding increasing noise corresponds to the forward diffusion process.

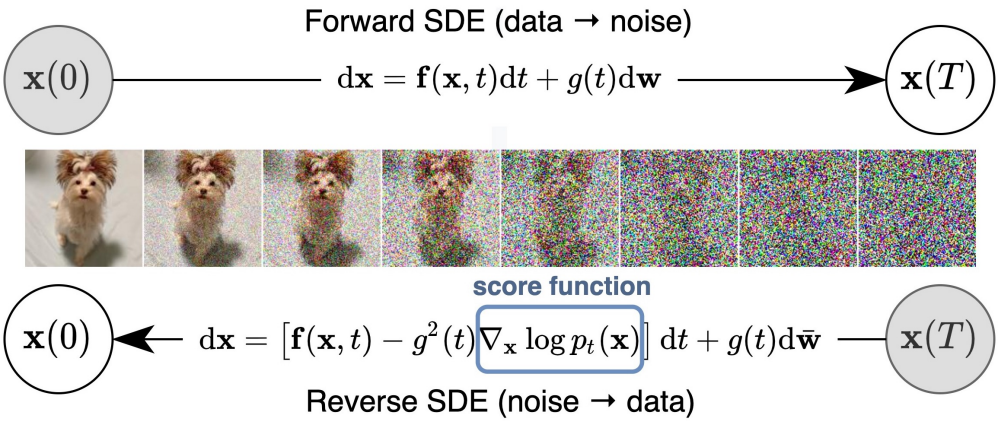
If we use the diffusion process annotation, the score approximates $\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$. Given a Gaussian distribution $\mathbf{x} \sim \mathcal{N}(\mu, \sigma^2\mathbf{I})$, we have :

$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \left(-\frac{1}{2\sigma^2}(\mathbf{x} - \boldsymbol{\mu})^2\right) = -\frac{\mathbf{x}-\boldsymbol{\mu}}{\sigma^2} = -\frac{\boldsymbol{z}}{\sigma}$ where $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Recall that $q(\mathbf{x}_t \mid \mathbf{x}_0) \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$ and therefore :

$$\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = \mathbb{E}_{q(\mathbf{x}_0)}\left[\nabla_{\mathbf{x}_t} q(\mathbf{x}_t \mid \mathbf{x}_0)\right] = \mathbb{E}_{q(\mathbf{x}_0)}\left[-\frac{\boldsymbol{z}_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}\right] = -\frac{\boldsymbol{z}_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

We can see that learning the score at each time step $\mathbf{s}_\theta(\mathbf{x}_t, t)$ is the same as learning the noise $\boldsymbol{z}_\theta(x_t, t)$



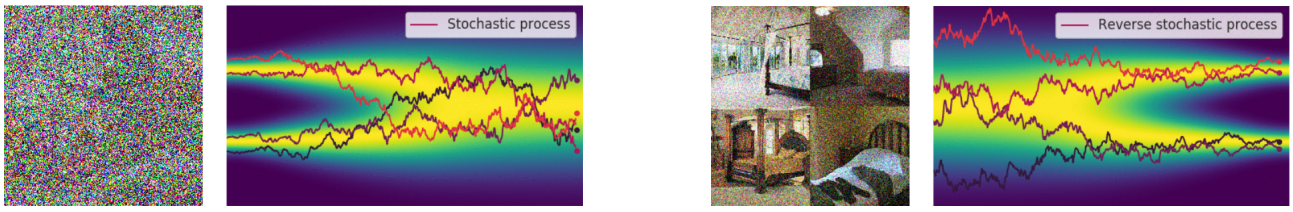
Process of Diffusion and Reverse Diffusion in Score-based Generative Models

We use stochastic differential equations (SDE) to noise our data. **SDE**s commonly take the following form :

$$\boxed{d\mathbf{X}_t = f(\mathbf{X}_t, t)\, dt + g(t)\, \mathbf{B}_t} \tag{3}$$

After that, we can denoise the data at each step with the reverse SDE (because $\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$) :

$$\boxed{d\mathbf{X}_t = \left[f(\mathbf{X}_t, t) - g^2(t)\nabla_x \log q(\mathbf{X}_t)\right]\, dt + g(t)\, \mathbf{B}_t} \tag{4}$$



Forward SDE Reverse SDE

FIGURE 7 – SDE : from Random Noise to Structured Representation with Score Functions

2.2.2 Pratical session

We restrict ourselves to a simple two-dimensional setting and want to generate a spiral distribution from a gaussian distribution.

— Forward Noising Process with the following SDE :

$$\mathrm{d}\mathbf{X}_t = -\frac{1}{2}\mathbf{X}_t \mathrm{d}t + \mathrm{d}\mathbf{B}_t, \quad \mathbf{X}_0 \sim \pi,$$

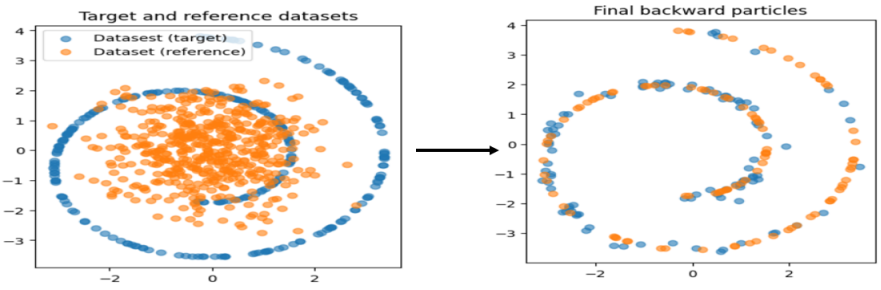
.

— Backward Denoising Process using the reverse SDE :

$$\mathrm{d}\mathbf{X}_t = \left(\frac{1}{2}\mathbf{X}_t + \nabla \log q_{T-t}(\mathbf{X}_t)\right) \mathrm{d}t + \mathrm{d}\mathbf{B}_t, \quad \mathbf{X}_0 \sim \mathcal{N},$$

.

We can then do forward noising process with the first equation, and reverse the process thanks to $\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ we learned.



Reverse Diffusion Process

3 Diffdock and Results

3.1 Diffdock, a diffusion model

DiffDock is a software containing several AI models, including diffusion models to simulate the binding process between ligands and proteins. By temporally reversing stochastic diffusion equations, DiffDock generates predictions about the optimal position and orientation of ligand molecules relative to the target protein. In diffusion models applied to images, the data that was noisy were pixels ; here, what we noise are the positions, orientations, and atom torsions. This process allows us to predict, from a protein and a ligand, the optimal binding site. At the beginning of the reverse process, ligands are initialized with random positions, orientations, and torsions, then by denoising step by step, the ligands converge towards an optimal position, orientation, and torsion.

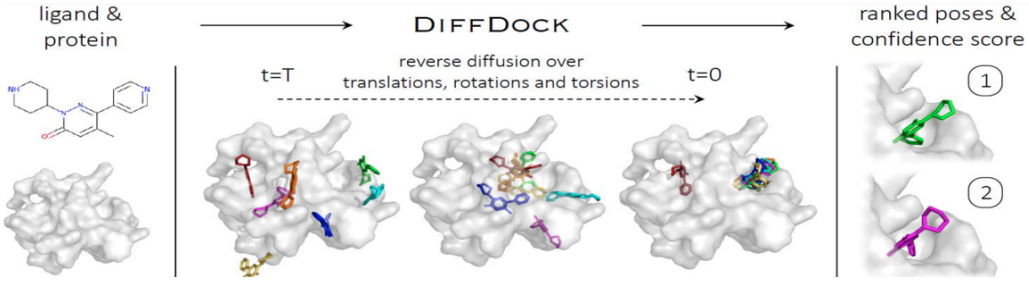
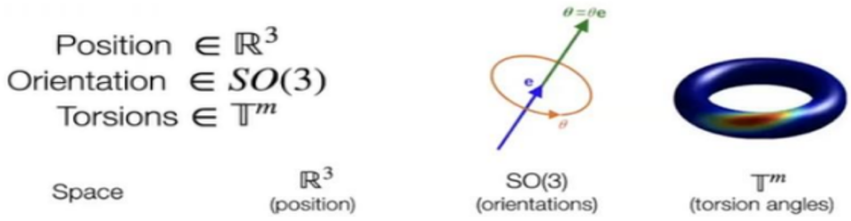


FIGURE 8 – Illustration of reverse diffusion on translations, rotations, and torsions

From a mathematical modeling perspective, we then work in the product space :

$$\mathbb{R}^3 \times SO(3) \times \mathbb{T}^m \quad \text{where} \quad \mathbb{T} = SO(2)$$

because,



A unique aspect of DiffDock is that it doesn't rely on a single diffusion model for predicting binding, but instead utilizes three independent models, which are trained simultaneously and completely independently. One model is used for predicting position only, a second model for predicting rotations, and a third model for predicting torsions. These three models are trained in parallel, yielding results for positions, orientations, and torsions, which are then concatenated to form our final binding prediction. [2]

We use the following SDE to generate independently position, orientation and torsion of the ligand on the protein :

$$d\mathbf{X}_t = \sqrt{\frac{d\sigma^2(t)}{dt}} d\mathbf{B}_t \quad \text{where} \quad \sigma^2 = \sigma^2_{\text{tr}}, \sigma^2_{\text{rot}}, \text{ or } \sigma^2_{\text{tor}} \quad \text{for } R^3, SO(3), \text{ and } T^m$$

3.2 DiffDock Confidence Score

In addition to the three parallel diffusion models for predicting optimal binding sites, the DiffDock software also offers a scoring model based on deep learning tools. During the training of the diffusion models, a scoring model is also trained, which determines the reliability of the results obtained. Specifically, this score is based on the root mean square distance (RMSD) between the prediction and the theoretical optimal site.

Other models exist to assess the reliability of binding predictions, such as GNINA. This model is based on deep learning methods and physical principles such as calculating the potential energy of the complex or molecular interactions to predict the affinity of a binding site.

The scoring model of GNINA is therefore based on calculating binding energies, such as the following :

- Van der Waals energy (Lennard-Jones equation) :

$$E_{\text{vdW}} = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^{6} \right] \tag{5}$$

- Electrostatic energy (Coulomb's equation) :

$$E_{\text{elec}} = \frac{1}{4\pi\epsilon_0} \frac{q_1 \cdot q_2}{r} \tag{6}$$

- Binding energy (sum of contributions) :

$$E_{\text{liaison}} = E_{\text{vdW}} + E_{\text{elec}} + \dots \tag{7}$$

- Total energy (energy of the ligand-protein complex) :

$$E_{\text{total}} = E_{\text{liaison}}^{\text{ligand-protein}} + E_{\text{internal}}^{\text{ligand}} \tag{8}$$

GNINA thus provides a deep learning model to calculate an affinity score based on the total energy of the protein-ligand complex. GNINA offers a physics-oriented approach to scoring the predicted binding site.

However, GNINA also offers an improvement system for predictions. Starting from a predicted binding site, with a docking model like DiffDock, GNINA minimization allows for locally modifying the prediction through gradient descent on position, orientation, and torsions towards a prediction that would minimize the potential energy of the site, or equivalently, maximize ligand-protein interactions.

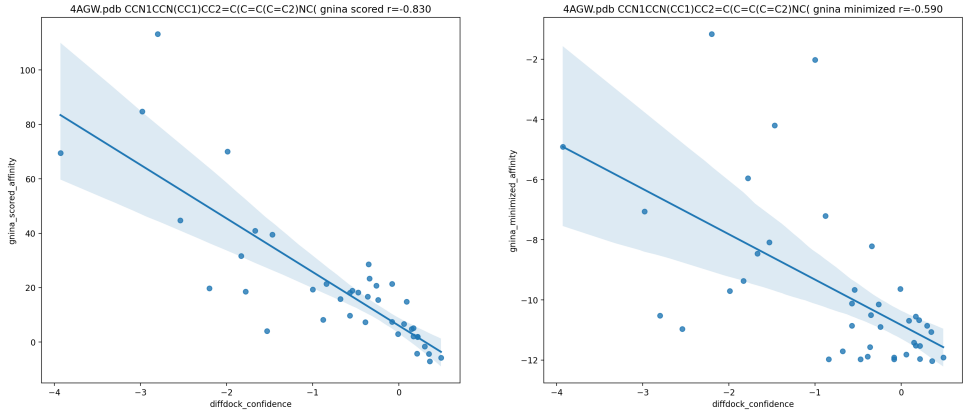
3.3 Results

DiffDock is an online accessible software on Google Colab and already trained. Therefore, we can apply the diffusion models to a case study to obtain the most interesting binding sites according to the DiffDock scoring model.[1]

3.3.1 DiffDock Confidence Score

We select the top 40 predictions with the highest DiffDock scores and plot their DiffDock score against their GNINA affinity. A higher DiffDock score indicates a more coherent prediction according to DiffDock, while a lower GNINA affinity suggests a more coherent prediction according to GNINA. We observe a correlation between the GNINA affinity and the DiffDock score, suggesting that when the prediction is reliable according to DiffDock, it is also reliable according to GNINA.

Next, we refine the predictions using the GNINA process towards local minima of potential energies, which locally moves the predicted binding sites towards a site that maximizes interactions between the protein and the ligand. We still observe the same correlation.



Comparaison of DiffDock Confidence with Gnina Predicted Affinity

3.3.2 Interactions graph

To analyze our results, we can focus on the interactions between the ligand and the protein when the ligand is placed according to the optimal prediction. We observe various types of bonds that ensure the stability of the ligand on the binding site, such as hydrogen bonds, Van der Waals interactions, and Pi-Stacking bonds.

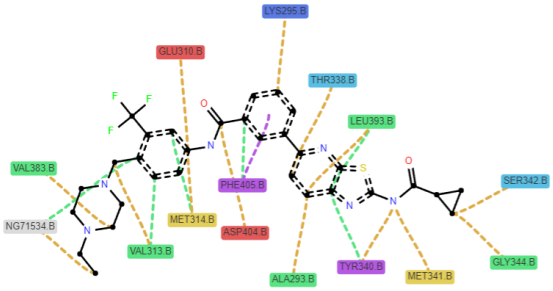


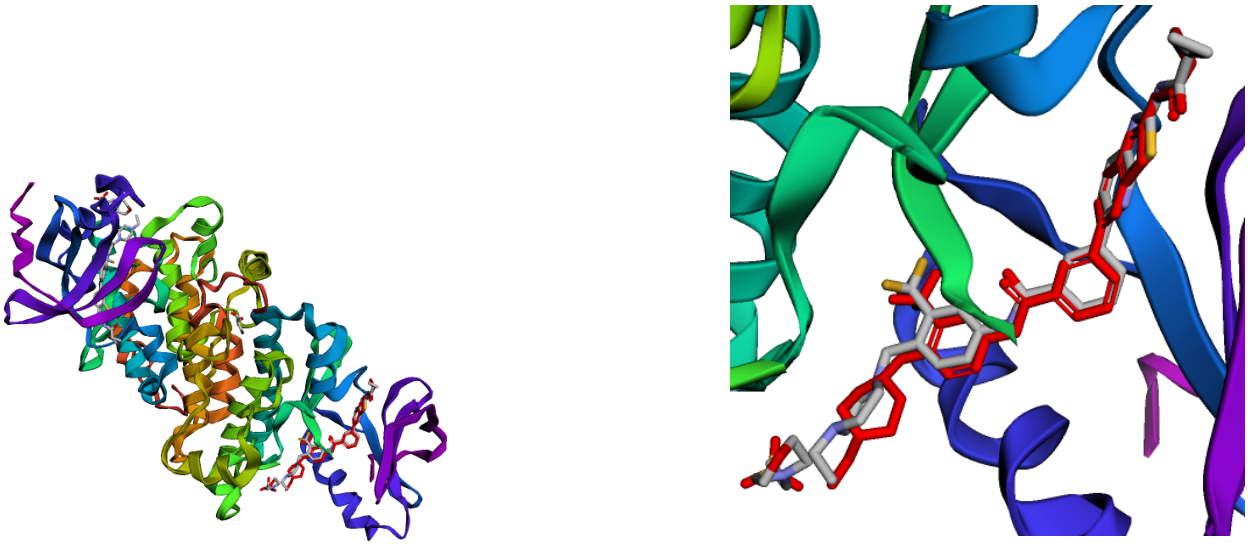
FIGURE 9 – Interactions graph

This interactions graph allows for a quantitative comparison of interactions in different protein-ligand complexes, thereby facilitating :

— **The analysis of binding modes :** Identifying how different ligands interact with the same binding sites on a protein, or comparing interactions within different conformations of the same protein.
— **Virtual screening :** Filtering and ranking potential ligands based on their similarity to known binding sites for active ligands, aiding in identifying promising candidates for experimental studies.
— **Rational drug design :** Guiding ligand modification to enhance binding affinity or selectivity by targeting identified key interactions.

3.3.3 Visualization of results

The obtained results can be easily visualized.



(a) The protein and its ligand

(b) Zoom on the predicted pose and the ground truth

FIGURE 10 – Best confidence score result achieved with DiffDock in our studied case

The results are very promising. As seen in the image, our predictions closely match the actual interactions between the ligand and the BCR-ABL protein. The binding position, torsion, and orientation of the ligand are nearly identical to the real structure, demonstrating the high accuracy and reliability of our approach. This level of precision enhances the confidence in using DiffDock in drug discovery.

3.4 Comparison with other molecular docking software

Conclusion

Références

[1] G. Corso. Diffdock : Diffusion steps, twists, and turns for molecular docking. `https://github.com/gcorso/DiffDock`.

[2] G. Corso, H. Stärk, B. Jing, R. Barzilay, and T. Jaakkola. Diffdock : Diffusion steps, twists, and turns for molecular docking. *arXiv*, 2017.

[3] Yang Song. Score-based generative modeling through stochastic differential equations. `https://yang-song.net/blog/2021/score/`, May 2021. Published on the blog of Yang Song.

[4] Lilian Weng. What are diffusion models ? `https://lilianweng.github.io/posts/2021-07-11-diffusion-models/`, July 2021.